Graphic/Display Performance Improve

Author: Brian Kang
Created: 4/9/2026~4/13/2026

Preface

This document provides methods to improve the existing “cam-thread” program.
It will cover the following topics:

Topic	description
zero-copy	“cam-thread” was implemented on CPU level. Now, we will use DMABUF and Fence logic.
HW composition	“cam-thread” using OpenGL ES to merge 4 input videos. We can use DRM Planes that can reduce OpenGL processing time.
Atomic Commit	“cam-thread” using the page-flip. We will use the atomic commit to Intricate and Subtle Processes.
multi-thread assign	“cam-thread” did not assign thread on CPU. We will learn how to assign the thread on the specific processor.
static video queue	“cam-thread” used malloc()/free() for the video queue. We will use the static method to solve the fragmentation issue.

“cam-thread” uses below 3 threads.

Thread	main framework	Description
VIN	V4L2	capture video from camera
DISPLAY	OpenGL ES	Draw video and GUI to backend (Wayland or DRM)
	Wayland	Embedded version of X11
	DRM	Direct Rendering Manager
IMR	IMR	For wide lenses distortion correction

We will use the following VideoQueue to move video information.

```
struct VideoQueue_s {
    int cam_idx;
    int buf_idx;
    int dmabuf_fd[3];
    int fence_fd[3];

    void *plane[3]; /* data_mem virtual address */
    uint32_t offset[3];
    uint32_t stride[3];
    int image_size;

    void *dispose_owner;
    void (*dispose)(void *arg); /* VideoQueue_t */
    struct VideoQueue_s *next;
};
typedef struct VideoQueue_s VideoQueue_t;

struct VideoQueueHeader_s {
   pthread_mutex_t lock;
   pthread_cond_t wait;
```

```
    pthread_cond_t cond_not_empty;
    pthread_cond_t cond_not_full;

     VideoQueue_t *front;
     VideoQueue_t *rear;
};
typedef struct VideoQueueHeader_s VideoQueueHeader_t;
```

The above structure shown is the structure before WORD alignment. It will be improved later.

VIN captures video frame, and sends to the DISPLAY thread. When DISPLAY thread draws the video, it needs to queue the frame back to VIN driver. It using callback function, dispose() within this structure.
If VIN send video frame to IMR, IMR have to return frame after it finish the distortion correction. And then IMR send the corrected video frame to DISPLAY. In this case DISPLAY thread has to send back the used frame to IMR. So, this structure also has information who need to process this video frame send back to driver.
On the DISPLAY thread, it can use cam_idx to decide the drawing location, and use plane[0] or dmabuf_fd[0] to get the data location or do it with DMA.

1. Current Method

It processes UYVY frames captured from the VIN as textures in OpenGL.
During development, a key requirement was to process UYVY directly to enable an "apples-to-apples comparison." While the formulas for YUV-to-RGB conversion are readily available, there are several factors that must be taken care for actual verification.
First, let's take a look at the basic formula.

BT601	Extension Y:16~235 CbCr:16~240	R = (255/219)*(Y-16) + (255/224)*1.402*(Cr-128) G = (255/219)*(Y-16) - (255/224)*1.772*(0.114/0.587)*(Cb-128) - (255/224)*1.402*(0.299/0.587)*(Cr-128) B = (255/219)*(Y-16) + (255/224)*1.772*(Cb-128)
	Compressed Y:0~255 CbCr:0~255	R = (Y) + 1.402*(Cr-128) G = (Y) - 1.772*(0.114/0.587)*(Cb-128) - 1.402*(0.299/0.587)*(Cr-128) B = (Y) + 1.772*(Cb-128)
BT701	Extension	R = (255/219)*(Y-16) + (255/224)*1.5748*(Cr-128) G = (255/219)*(Y-16) - (255/224)*1.8556*(0.0722/0.7152)*(Cb-128) - (255/224)*1.5748*(0.2126/0.7152)*(Cr-128) B = (255/219)*(Y-16) + (255/224)*1.8556*(Cb-128)
	Compressed	R = Y + 1.5748*(Cr-128) G = Y - 1.8556*(0.0722/0.7152)*(Cb-128) - 1.5748*(0.2126/0.7152)*(Cr-128) B = Y + 1.8556*(Cb-128)

The following is a simpler version for actual use.

BT601	Extension Y:16~235 CbCr:16~240	R = 1.164*(Y-16) + 1.596*(Cr-128) G = 1.164*(Y-16) - 0.392*(Cb-128) - 0.813*(Cr-128) B = 1.164*(Y-16) + 2.017*(Cb-128)
	Compressed Y:0~255 CbCr:0~255	R = (Y) + 1.402*(Cr-128) G = (Y) - 0.344*(Cb-128) - 0.714*(Cr-128) B = (Y) + 1.772*(Cb-128)
BT701	Extension	R = 1.1644*(Y-16) + 1.7928*(Cr-128) G = 1.1644*(Y-16) - 0.2133*(Cb-128) - 0.5330*(Cr-128) B = 1.1644*(Y-16) + 2.1125*(Cb-128)
	Compressed	R = Y + 1.5748*(Cr-128) G = Y - 0.1873*(Cb-128) - 0.4681*(Cr-128) B = Y + 1.8556*(Cb-128)

Vertical Shader commonly uses the following glsl code.

```
char vShaderStr[] =
    "#version 310 es                            \n"
    "layout(location = 0) in vec4 a_position;   \n"
    "layout(location = 1) in vec2 a_texCoord;   \n"
    "out vec2 v_texCoord;                       \n"
    "void main() {                              \n"
    "   gl_Position = a_position;               \n"
    "   v_texCoord = a_texCoord;                \n"
    "}
```

If we write the shader based on the previously mentioned, simpler version, it looks like this:

BT601 Extension	`char fShaderStr_UYVY[] =`
	`"#version 310 es\n"`
	`"precision mediump float;  \n"`
	`"in vec2 v_texCoord;  \n"`
	`"layout(location = 0) out vec4 FragColor;  \n"`
	`"uniform sampler2D s_textureY;  \n"`
	`"uniform sampler2D s_textureUV;  \n"`
	`"void main() {  \n"`
	`"  float r, g, b, y, u, v;  \n"`
	`"  y = texture(s_textureY,  v_texCoord).g - 0.0625;\n"`
	`"  u = texture(s_textureUV, v_texCoord).r - 0.5;  \n"`
	`"  v = texture(s_textureUV, v_texCoord).b - 0.5;  \n"`
	`"  r = 1.164 * y  + 1.596 * v;  \n"`
	`"  g = 1.164 * y - 0.392 * u - 0.813 * v;  \n"`
	`"  b = 1.164 * y + 2.017 * u;  \n"`
	`"  FragColor = vec4(r, g, b, 1.0);  \n"`
	`"}\n";`

BT601 Compressed	`char fShaderStr_UYVY[] =`
	`"#version 310 es\n"`
	`"precision mediump float;  \n"`
	`"in vec2 v_texCoord;  \n"`
	`"layout(location = 0) out vec4 FragColor;  \n"`
	`"uniform sampler2D s_textureY;  \n"`
	`"uniform sampler2D s_textureUV;  \n"`
	`"void main() {  \n"`
	`"  float r, g, b, y, u, v;  \n"`
	`"  y = texture(s_textureY,  v_texCoord).g;  \n"`
	`"  u = texture(s_textureUV, v_texCoord).r - 0.5;\n"`
	`"  v = texture(s_textureUV, v_texCoord).b - 0.5;\n"`
	`"  r = y  + 1.402 *v;  \n"`
	`"  g = y - 0.344136 * u - 0.714136 * v;  \n"`
	`"  b = y + 1.772 * u;  \n"`
	`"  FragColor = vec4(r, g, b, 1.0);  \n"`
	`"}\n";`

BT701 Extension	`char fShaderStr_UYVY[] =`
	`"#version 310 es\n"`
	`"precision mediump float;  \n"`
	`"in vec2 v_texCoord;  \n"`
	`"layout(location = 0) out vec4 FragColor;  \n"`
	`"uniform sampler2D s_textureY;  \n"`
	`"uniform sampler2D s_textureUV;  \n"`
	`"void main() {  \n"`

	`"  float r, g, b, y, u, v;  \n"`
	`"  y = texture(s_textureY,  v_texCoord).g - 0.0625;\n"`
	`"  u = texture(s_textureUV, v_texCoord).r - 0.5;  \n"`
	`"  v = texture(s_textureUV, v_texCoord).b - 0.5;  \n"`
	`"  r = 1.1644*y  + 1.7928 * v;  \n"`
	`"  g = 1.1644*y - 0.2133 * u - 0.5330 * v;  \n"`
	`"  b = 1.1644*y + 2.1125 * u;  \n"`
	`"  FragColor = vec4(r, g, b, 1.0);  \n"`
	`"}\n";`

BT701
Compressed
	`char fShaderStr_UYVY[] =`
	`"#version 310 es\n"`
	`"precision mediump float;  \n"`
	`"in vec2 v_texCoord;  \n"`
	`"layout(location = 0) out vec4 FragColor;  \n"`
	`"uniform sampler2D s_textureY;  \n"`
	`"uniform sampler2D s_textureUV;  \n"`
	`"void main() {  \n"`
	`"  float r, g, b, y, u, v;  \n"`
	`"  y = texture(s_textureY,  v_texCoord).g;  \n"`
	`"  u = texture(s_textureUV, v_texCoord).r - 0.5;\n"`
	`"  v = texture(s_textureUV, v_texCoord).b - 0.5;\n"`
	`"  r = y  + 1.5748 * v;  \n"`
	`"  g = y - 0.1873 * u - 0.4681 * v;  \n"`
	`"  b = y + 1.8556 * u;  \n"`
	`"  FragColor = vec4(r, g, b, 1.0);  \n"`
	`"}\n";`

BT470
PAL
	`char fShaderStr_UYVY[] =`
	`"#version 310 es\n"`
	`"precision mediump float;  \n"`
	`"in vec2 v_texCoord;  \n"`
	`"layout(location = 0) out vec4 FragColor;  \n"`
	`"uniform sampler2D s_textureY;  \n"`
	`"uniform sampler2D s_textureUV;  \n"`
	`"void main() {  \n"`
	`"  float r, g, b, y, u, v;  \n"`
	`"  y = texture(s_textureY,  v_texCoord).g;  \n"`
	`"  u = texture(s_textureUV, v_texCoord).r - 0.5;\n"`
	`"  v = texture(s_textureUV, v_texCoord).b - 0.5;\n"`
	`"  r = y  + 1.13983 * v;  \n"`
	`"  g = y - 0.39465 * u - 0.58060 * v;  \n"`
	`"  b = y + 2.03211 * u;  \n"`
	`"  FragColor = vec4(r, g, b, 1.0);  \n"`
	`"}\n";`

In the above fragment shaders, you can see two `uniform sampler2D` variables, `s_textureY` and `s_textureUV . The reason for this is, when representing two pixels on the UYVY format, a single UV value is shared between every two Y pixels . Consequently, the following processing steps are required for output generation.

```
GLuint init_shader_UYVY(void)
{
    char vShaderStr[] = { /* use upper v-shader method */ };
    char fShaderStr_UYVY[] = { /* select one of upper f-shader method */};
    GLuint programObject;
    GLint location;
    int ret;
```

```
    ret = init_shader_program(__func__, &programObject, vShaderStr,fShaderStr_UYVY);
    if(ret)
        return GL_FALSE;
    glUseProgram(programObject);
    location = glGetUniformLocation ( programObject, "s_textureY" );
    if(location == -1) {
        printf("Err location. s_textureY\n");
        return GL_FALSE;
    }
    glUniform1i(location, 0);

    location = glGetUniformLocation ( programObject, "s_textureUV" );
    if(location == -1) {
        printf("Err location. s_textureUV\n");
        return GL_FALSE;
    }
    glUniform1i(location,1);

    check_gl_error();
    return programObject;
}
```

The following code is to create the texture.

```
static int texture_prepare_gl(struct texture * ptex, const char *fname)
{
  GLuint texture_id;
  GLenum texture_format;
  int w = ptex->w;
  int h = ptex->h;

  /* for V4L2_PIX_FMT_UYVY */
  glGenTextures(1, &texture_id);
  ptex->id  = texture_id;
  glBindTexture(GL_TEXTURE_2D, texture_id);
  glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR); //or GL_NEAREST
  glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR); //or GL_NEAREST
  glTexImage2D(GL_TEXTURE_2D, 0,
#ifdef USE_GLES2
       GL_LUMINANCE, w,h,0, GL_LUMINANCE,   /* GL_RG_EXT */
#else
       GL_RG, w, h, 0, GL_RG,
#endif
       GL_UNSIGNED_BYTE, NULL);

  glGenTextures(1, &texture_id);
  ptex->id1  = texture_id;
  glBindTexture(GL_TEXTURE_2D, texture_id);
  glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR); //or GL_NEAREST
  glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR); //or GL_NEAREST
  glTexImage2D(GL_TEXTURE_2D, 0, GL_LUMINANCE,     //GL_RGBA,
       w / 2, h, 0, GL_LUMINANCE,                                //GL_RGBA,
       GL_UNSIGNED_BYTE, NULL);

  return 0;
}
```

Y is stored as two bytes, utilizing the second byte, and is saved to "ptex->id".

UV is stored as 4 bytes, with U occupying the first byte and V the third; it is stored in `ptex->id1`. In this case, it used the halved width value.

The below example code uses the data_mem that received from a camera to update the texture data .

```
/*
use plane[0] address on data_mem.
*/
void texture_update_from_buffer(struct texture *tex, unsigned char *data_mem)
{
  int w = tex->w;
  int h = tex->h;

  /* for y_plane */
  glActiveTexture(GL_TEXTURE0);
  glBindTexture (GL_TEXTURE_2D, tex->id);  /* y_plane */
    glTexImage2D(GL_TEXTURE_2D, 0,
#ifdef USE_GLES2
     GL_RG_EXT, w, h, 0, GL_RG_EXT,
#else
     GL_RG, w, h, 0, GL_RG,
#endif
     GL_UNSIGNED_BYTE, data_mem);

  /* for uv_plane */
  glActiveTexture(GL_TEXTURE1);
  glBindTexture (GL_TEXTURE_2D, tex->id1); /* uv_plane */
  glTexImage2D(GL_TEXTURE_2D, 0,
    GL_RGBA,
    w / 2, h, 0,
    GL_RGBA,
    GL_UNSIGNED_BYTE, data_mem);

  check_gl_error();
}
```

Let me explain one step deeper. On the glsl, R is first, G is second, B is third, and A is fourth location.

Y	We use GL_RG, which is 2 bytes in size, and the Y value is located at the G position.
UV	We use GL_RGBA, which is 4 bytes in size, with the U value located at the R position and the V value at the B position.

You will find that with just a minor modification, the YUYV format can also be supported.

Note: If your system concerns about video quality, use this direct conversion method, or you have to check VIN driver and OpenGL Extension source code. On R-CAR VIN driver, HW can support all, but it is fixed as BT601 Extension mode.

The following example using mod() to extract U and V values on a single texture.

```
void main()
{
    // 1. Sampling: GPU reads data as LUMINANCE_ALPHA
    // la.r = Luminance(Y), la.a = Alpha(U or V)
    vec4 la = texture2D(u_texture, v_texCoord);

    float y, u, v;

    // Since UYVY follows the [U0 Y0 V0 Y1] structure,
    // the Luminance channel (la.r) always maps to the Y value.
    y = la.r;

    // 2. U/V Extraction (Chroma sampling position calculation)
```

```
    // Determine U or V by checking if the current pixel's X coordinate is even or odd.
    // Since u_texture width is required, passing it as a uniform is recommended.
    // Hardcoded logic is used here for simplicity.

    // [Expert Advice] Even/Odd pixel determination logic can impact shader performance.
    // The simplest implementation is to use la.a as the average of U and V,
    // but this may blur colors.
    // For precise reproduction, we utilize the pixel's X coordinate.

    float pixel_x = v_texCoord.x * 1280.0; // Example width: 1280
    if (mod(floor(pixel_x), 2.0) < 0.5) {
        // Even pixel: la.a is the U value
        u = la.a;
        // Fetch la.a from the next pixel to use as V (color correction)
        v = texture2D(u_texture, v_texCoord + vec2(1.0/1280.0, 0.0)).a;
    } else {
        // Odd pixel: la.a is the V value
        v = la.a;
        // Fetch la.a from the previous pixel to use as U (color correction)
        u = texture2D(u_texture, v_texCoord - vec2(1.0/1280.0, 0.0)).a;
    }

    // 3. Range Adjustment (YUV is normalized between 0.0 and 1.0)
    y = (y - 16.0/255.0);
    u = (u - 128.0/255.0);
    v = (v - 128.0/255.0);

    // 4. RGB Conversion
    vec3 yuv = vec3(y, u, v);
    vec3 rgb = yuv2rgb_matrix * yuv;

    gl_FragColor = vec4(rgb, 1.0);
}
```

2. OpenGL extension for UYVY

The chapter describes how to draw UYVY using the **EGLImage object** feature in OpenGL Extensions.
I wanted to explain the data_mem address method and dmabuf method, but could not find the data_mem solution. So, this will explain only dmabuf method.
This method is based on below link,
https://github.com/renesas-rz/demo-camera-image-overlay/tree/main/raw-video-to-lcd

This code using below structure.

```
struct {
  int dmabuf_fd;      /* FD of dmabuf */
  char *p_virt_addr;  /* virtual address space address */
  size_t size;        /* Buffer's Size */
  int fence_fd;       /* fence fd for dma */
} v4l2_dmabuf_exp_t;
```

At first, you have to get address for EGL extension functions.

```
#include <assert.h>
#include <stdlib.h>

#include <drm/drm_fourcc.h>

#include "egl.h"
#include "util.h"

/*****************************************************************************
 *                              EGL EXTENSIONS                               *
 *****************************************************************************/

typedef EGLImageKHR (*EGLCREATEIMAGEKHR) (EGLDisplay dpy,
                                          EGLContext ctx,
                                          EGLenum target,
                                          EGLClientBuffer buffer,
                                          EGLint * p_attr_list);

typedef EGLBoolean (*EGLDESTROYIMAGEKHR) (EGLDisplay dpy, EGLImageKHR image);

EGLCREATEIMAGEKHR  eglCreateImageKHR;
EGLDESTROYIMAGEKHR eglDestroyImageKHR;

bool egl_init_ext_funcs(EGLDisplay display)
{
    /* Check parameter */
    assert(display != EGL_NO_DISPLAY);

    /* Get address of function 'eglCreateImageKHR' */
    eglCreateImageKHR = (EGLCREATEIMAGEKHR)
                        eglGetProcAddress("eglCreateImageKHR");

    /* Get address of function 'eglDestroyImageKHR' */
    eglDestroyImageKHR = (EGLDESTROYIMAGEKHR)
                         eglGetProcAddress("eglDestroyImageKHR");

    if ((eglCreateImageKHR == NULL) || (eglDestroyImageKHR == NULL) ||
        !egl_is_ext_supported(display, "EGL_KHR_image_base") ||
        !egl_is_ext_supported(display, "EGL_EXT_image_dma_buf_import"))
    {
```

```
        printf("Error: Failed to init EGL extension functions\n");
        return false;
    }

    return true;
}
```

```
typedef void (*GLEGLIMAGETARGETTEXTURE2DOES) (GLenum target,
                                              GLeglImageOES image);

GLEGLIMAGETARGETTEXTURE2DOES glEGLImageTargetTexture2DOES;


bool gl_init_ext_funcs()
{
    /* Get address of function 'glEGLImageTargetTexture2DOES' */
    glEGLImageTargetTexture2DOES = (GLEGLIMAGETARGETTEXTURE2DOES)
                            eglGetProcAddress("glEGLImageTargetTexture2DOES");

    if ((glEGLImageTargetTexture2DOES == NULL) ||
        !gl_is_ext_supported("GL_OES_surfaceless_context") ||
        !gl_is_ext_supported("GL_OES_EGL_image_external") ||
        !gl_is_ext_supported("GL_OES_EGL_image_external_essl3") ||
        !gl_is_ext_supported("GL_EXT_YUV_target"))
    {
        printf("Error: Failed to init OpenGL ES extension functions\n");
        return false;
    }

    return true;
}
```

Next step is creating EGL Image object.

The EGL Image will be generated 5 or 6 that have the same number of video frame on VIN.

And dmabuf_fd will be provided from VIN module.

```
EGLImageKHR egl_create_yuyv_image(EGLDisplay display, uint32_t width,
                                  uint32_t height, int dmabuf_fd)
{
    EGLImageKHR img = EGL_NO_IMAGE_KHR;

    /* Check parameters */
    assert(display != EGL_NO_DISPLAY);
    assert((width > 0) && (height > 0) && (dmabuf_fd > 0));

    EGLint img_attribs[] =
    {
        /* The logical dimensions of YUYV buffer in pixels */
        EGL_WIDTH, width,
        EGL_HEIGHT, height,

        /* Pixel format of the buffer, as specified by 'drm_fourcc.h' */
        EGL_LINUX_DRM_FOURCC_EXT, DRM_FORMAT_YUYV,

        /* The dmabuf file descriptor of plane 0 of the image */
        EGL_DMA_BUF_PLANE0_FD_EXT, dmabuf_fd,
```

```
        /* The offset from the start of the dmabuf of the first sample in
         * plane 0, in bytes */
        EGL_DMA_BUF_PLANE0_OFFSET_EXT, 0,

        /* The number of bytes between the start of subsequent rows of samples
         * in plane 0 */
        EGL_DMA_BUF_PLANE0_PITCH_EXT, width * 2, /* 2 bytes per pixel */

        /* Y, U, and V color range from [0, 255] */
        EGL_SAMPLE_RANGE_HINT_EXT, EGL_YUV_FULL_RANGE_EXT,

        /* The chroma samples are sub-sampled only in horizontal dimension,
         * by a factor of 2 */
        EGL_YUV_CHROMA_VERTICAL_SITING_HINT_EXT, EGL_YUV_CHROMA_SITING_0_EXT,
        EGL_YUV_CHROMA_HORIZONTAL_SITING_HINT_EXT,
                                                EGL_YUV_CHROMA_SITING_0_5_EXT,
        EGL_NONE,
    };

    /* Create EGLImage from a Linux dmabuf file descriptor */
    img = eglCreateImageKHR(display, EGL_NO_CONTEXT, EGL_LINUX_DMA_BUF_EXT,
                            (EGLClientBuffer)NULL, img_attribs);
    if (img == EGL_NO_IMAGE_KHR)
    {
        printf("Error: Failed to create YUYV EGLImage\n");
    }

    return img;
}
```

eglCreateImage is used to create an **EGLImage object** from an existing image resource *buffer.*
On the attribute, we can select colorspace and color range.

Key	Value	related V4L2 colorspace
EGL_YUV_COLOR_SPACE_HINT_EXT	EGL_ITU_REC601_EXT	V4L2_COLORSPACE_SMPTE170M or V4L2_COLORSPACE_SRGB
	EGL_ITU_REC709_EXT	V4L2_COLORSPACE_REC709
	EGL_ITU_REC2020_EXT	V4L2_COLORSPACE_BT2020

Key	Value	color range
EGL_SAMPLE_RANGE_HINT_EXT	EGL_YUV_FULL_RANGE_EXT	[0, 255]
	EGL_YUV_NARROW_RANGE_EXT	Y: [16~235], UV: [16~240]

Create a number of EGL Image objects equal to the number of frames used by the VIN, and store them in `p_imgs[]` on egl_create_yuyv_images().

Below is an example to create texture from EGLImage object.

```
GLuint gl_create_external_texture(EGLImageKHR image)
{
    GLuint texture = 0;

    /* Check parameter */
    assert(image != EGL_NO_IMAGE_KHR);

    /* Use external texture for special image layouts, such as: YUYV, NV12... */
```

```
    glGenTextures(1, &texture);
    glBindTexture(GL_TEXTURE_EXTERNAL_OES, texture);

    /* Min filter and mag filter should be set to GL_NEAREST.
     * The output quality may be affected if these are set to GL_LINEAR */
    glTexParameteri(GL_TEXTURE_EXTERNAL_OES, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
    glTexParameteri(GL_TEXTURE_EXTERNAL_OES, GL_TEXTURE_MAG_FILTER, GL_NEAREST);

    /* In GL_TEXTURE_EXTERNAL_OES, only GL_CLAMP_TO_EDGE is accepted as
     * GL_TEXTURE_WRAP_S and GL_TEXTURE_WRAP_T */
    glTexParameteri(GL_TEXTURE_EXTERNAL_OES, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
    glTexParameteri(GL_TEXTURE_EXTERNAL_OES, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);

    /* Define texture from an existing EGLImage */
    glEGLImageTargetTexture2DOES(GL_TEXTURE_EXTERNAL_OES, image);

    /* Unbind texture */
    glBindTexture(GL_TEXTURE_EXTERNAL_OES, 0);

    return texture;
}
```

In this case, too, gl_create_external_textures() creates number of texture equal to the number of frames used by the VIN.

When we render texture, using below example that uses glBindTexture().

```
void gl_render_texture(GLuint prog, GLenum target, GLuint tex, gl_res_t res)
{
    GLint tmp_cnt = 0;

    /* Check parameter */
    assert((prog != 0) && (tex != 0));

    /* Use program object for rendering texture 'tex' */
    glUseProgram(prog);

    /* Enable attribute 0 since it's disabled by default */
    glEnableVertexAttribArray(0);

    /* Show OpenGL ES how the vertex array should be interpreted */
    glBindBuffer(GL_ARRAY_BUFFER, res.vbo_canvas_verts);

    glVertexAttribPointer(0, 4, GL_FLOAT, GL_FALSE,
                          4 * sizeof(GLfloat), (void *)0);

    /* Bind the texture */
    glBindTexture(target, tex);

    /* Render the texture */
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, res.ibo);
    glGetBufferParameteriv(GL_ELEMENT_ARRAY_BUFFER, GL_BUFFER_SIZE, &tmp_cnt);

    glDrawElements(GL_TRIANGLES, tmp_cnt, GL_UNSIGNED_BYTE, 0);

    /* Wait until 'glDrawElements' finishes */
    glFinish();

    /* Unbind texture */
    glBindTexture(target, 0);
```

```
    /* Unbind buffers */
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0);

    /* Disable attribute */
    glDisableVertexAttribArray(0);
}
```

Summary

item	Basic	with DMABUF
data copy	glTexImage2D().	No.
texture target in glBindTexture()	GL_TEXTURE_2D	GL_TEXTURE_EXTERNAL_OES
shader sampler	sampler2D	samplerExternalOES
Performance	Increase CPU/BUS load on High Resolution.	Zero-copy

Below is a shader example for YUVY input.

vertex shader

```
layout (location = 0) in vec4 aVertex; /* <vec2 pos, vec2 tex> */

out vec2 yuyvTexCoord;

void main(void)
{
    /* Just convert a 2D coordinate to 3D coordinate */
    gl_Position = vec4(aVertex.xy, 0.0, 1.0);

    /* Pass texture coordinate to fragment shader */
    yuyvTexCoord = aVertex.zw;
}
```

fragment shader

```
#version 300 es
#extension GL_EXT_YUV_target : require

/* Copyright (c) 2023 Renesas Electronics Corp.
 * SPDX-License-Identifier: MIT-0 */

precision mediump float;

uniform __samplerExternal2DY2YEXT yuyvTexture;

in vec2 yuyvTexCoord;
out vec4 FragColor;

void main(void)
{
    /* Re-adjust texture coordinate */
    vec2 flippedTexCoord = vec2(yuyvTexCoord.x, 1.0 - yuyvTexCoord.y);

    /* Get color of texture */
    vec3 texColor = vec3(texture(yuyvTexture, flippedTexCoord));

    /* Convert 'texColor' from YUV (full range) to RGB format */
    FragColor = vec4(yuv_2_rgb(texColor, itu_601_full_range), 1.0);
}
```

Below is an main() example that using dmabuf with YUVY format.

```
#define YUYV_BUFFER_COUNT 5  /* or 6 */
int main()
{
    //-------------------------------------------------------
    // step 1: Setup interrupt signal handler
    //-------------------------------------------------------

    //-------------------------------------------------------
    // step 2: setup v4l2 device
    //-------------------------------------------------------
    cam_fd = v4l2_open_dev(opt.cam_dev);
    v4l2_set_format(cam_fd, opt.width, opt.height, V4L2_PIX_FMT_YUYV, V4L2_FIELD_NONE);

    //-------------------------------------------------------
    // step 3: Allocate YUYV(UYVY) buffers for from V4L2 device
    //-------------------------------------------------------
    buf_cnt = YUYV_BUFFER_COUNT;
    p_yuyv_bufs = v4l2_alloc_dmabufs(cam_fd, &buf_cnt);

    //-------------------------------------------------------
    // step 4: SET UP WAYLAND
    //-------------------------------------------------------
    p_wl_display = wl_connect_display();
    p_wl_window = wl_create_window(p_wl_display, WINDOW_TITLE,
                                   opt.width, opt.height);

    //-------------------------------------------------------
    // step 5: Create EGL display, context, and window surface
    //-------------------------------------------------------
    egl_display = egl_connect_display((EGLNativeDisplayType)p_wl_display->p_display,
                      &egl_config);
    egl_surface = eglCreateWindowSurface(egl_display, egl_config,
                                         (EGLNativeWindowType)
                                         p_wl_window->p_egl_window, NULL);
    egl_context = egl_create_context(egl_display, egl_config, egl_surface);

    //-------------------------------------------------------
    // step 6: Setup OpenGL ES
    //-------------------------------------------------------
    rec_prog = gl_create_prog_from_src("rectangle.vs.glsl", "rectangle.fs.glsl");
    conv_prog = gl_create_prog_from_src("yuyv-to-rgb.vs.glsl", "yuyv-to-rgb.fs.glsl");
    text_prog = gl_create_prog_from_src("text.vs.glsl", "text.fs.glsl");
    gl_res = gl_create_resources(opt.width, opt.height, FONT_FILE);
    gl_init_ext_funcs(); /* get eglCreateImageKHR, eglDestroyImageKHR */

    //-------------------------------------------------------
    // step 7: Create EGLImages from the YUYV(UYVY) buffers
    //-------------------------------------------------------
    p_yuyv_imgs = egl_create_yuyv_images(egl_display, opt.width, opt.height,
                        p_yuyv_bufs, YUYV_BUFFER_COUNT);
    p_yuyv_texs = gl_create_external_textures(p_yuyv_imgs, YUYV_BUFFER_COUNT);

    //-------------------------------------------------------
    // step 8: Prepare Capture data
    //-------------------------------------------------------
    v4l2_enqueue_bufs(cam_fd, YUYV_BUFFER_COUNT); /* ENQUE */
    v4l2_enable_capturing(cam_fd);                /* STREAM_ON */

    //-------------------------------------------------------
    // step 9: Main loop
    //-------------------------------------------------------
```

```
    while (true)
    {
        ret = wl_display_dispatch_pending(p_wl_display->p_display);
        v4l2_dequeue_buf(cam_fd, &cam_buf);  /* DEQUE */


        gl_render_texture(conv_prog, GL_TEXTURE_EXTERNAL_OES,
                          p_yuyv_texs[cam_buf.index], gl_res);

        //--gl_draw_rectangle(rec_prog, gl_res);
        //--gl_draw_text(text_prog, "This is a text", 25.0f, 25.0f, BLACK, gl_res);
        eglSwapBuffers(egl_display, egl_surface);

        /* Render the filled buffer */
        gl_render(p_yuyv_texs[filled_buf_index], rec_prog, conv_prog, text_prog, gl_res);
        eglSwapBuffers(egl_display, egl_surface);

        /* Re-enqueue the buffer to be filled again */
        v4l2_enqueue_buf(cam_fd, cam_buf.index); /* ENQUE */
    }

    //-------------------------------------------------------
    //  STEP 10: STOP CAPTURING DATA
    //-------------------------------------------------------
    v4l2_disable_capturing(cam_fd);

    //-------------------------------------------------------
    // STEP 11: CLEAN UP OPENGL ES
    //-------------------------------------------------------
    gl_delete_resources(gl_res);

    glDeleteProgram(rec_prog);
    glDeleteProgram(conv_prog);
    glDeleteProgram(text_prog);

    //-------------------------------------------------------
    //  STEP 12: CLEAN UP EGL
    //-------------------------------------------------------
    eglDestroyContext(egl_display, egl_context);
    eglDestroySurface(egl_display, egl_surface);
    egl_disconnect_display(egl_display);

    //-------------------------------------------------------
    // STEP 13: CLEAN UP WAYLAND
    //-------------------------------------------------------
    wl_delete_window(p_wl_window);
    wl_disconnect_display(p_wl_display);

    //-------------------------------------------------------
    // STEP 14: CLEAN UP V4L2 DEVICE
    //-------------------------------------------------------
    gl_delete_textures(p_yuyv_texs, YUYV_BUFFER_COUNT);
    egl_delete_images(egl_display, p_yuyv_imgs, YUYV_BUFFER_COUNT);
    v4l2_dealloc_dmabufs(p_yuyv_bufs, YUYV_BUFFER_COUNT);
    close(cam_fd);

    return 0;
}
```

3 DMA fence

DMA is to transfer of data from source memory to target memory without any interference of CPU. Specifically, we transfer data from the CPU memory allocated by VIN to the GPU memory allocated by EGLImage. If `glBindTexture()` executes too rapidly—such that read operations are initiated before the DMA transfer is complete—screen tearing will occur.
To resolve this issue, we employ fence logic; this functions as an "out-fence" from the perspective of VIN and an "in-fence" from the perspective of the GPU.

3.1 create out-fence on VIN QBUF

```
#include <linux/videodev2.h>
#include <sys/ioctl.h>
#include <unistd.h>
#include <stdio.h>

// to generate the fence fd that indicates the 음 done state.
int qbuf_with_fence(struct vin_data *vin, int cam_idx, int buf_idx)
{
    struct vin_device *vdev = &vin->dev[cam_idx];
    VideoQueue_t *vqueue = &vdev->pool[buf_idx];
    struct v4l2_buffer buf = {0};

    buf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
    buf.memory = V4L2_MEMORY_DMABUF;
    buf.index = index;
    buf.flags = V4L2_BUF_FLAG_OUT_FENCE; /* to create fence */

    if (ioctl(vdev->fd, VIDIOC_QBUF, &buf) < 0) {
        perror("VIDIOC_QBUF failed");
        return -1;
    }

    // This file descriptor has a signal when DNA is done
    // From the recipient's perspective, this becomes 'In-fence'.
    vqueue->fence_fd = buf.request_fd;
    return 0;
}
```

It needs close(fence_fd) when the wait process is done on the receiver side.

3.2 On OpenGL

To use the fence_fd, you must implement the Sync Object mechanism on OpenGL/EGL.
This is the process of inserting a command into the GPU pipeline—specifically, "wait until the camera finishes writing to memory"—before the GPU begins rendering.
Use EGL_ANDROID_native_fence_sync extension function to convert fence_fd to EGL Sync object, and then put it on the GPU server wait stream (glWaitSyncAPPLE or eglWaitSyncKHR). This method is developed on Android, and EGL using same name.

```
#include <EGL/egl.h>
#include <EGL/eglext.h>
```

```
#include <GLES2/gl2.h>
#include <GLES2/gl2ext.h>

/**
* @param buffer_index: The index of the buffer to be rendered
*  (out of the 6-buffer static queue)
* @param fence_fd: DMA completion fence FD received from V4L2 (VIN)
*/
void draw_camera_frame_with_fence(int buffer_index, int fence_fd) {
    EGLDisplay dpy = eglGetCurrentDisplay();

    // 1. In-fence Handling (Ensures GPU waits before rendering)
    if (fence_fd != -1) {
        // Create an EGL Sync object from the native fence_fd
        EGLint attribs[] = { EGL_SYNC_NATIVE_FENCE_FD_ANDROID, fence_fd, EGL_NONE };
        EGLSyncKHR sync = eglCreateSyncKHR(dpy, EGL_SYNC_NATIVE_FENCE_ANDROID, attribs);

        if (sync != EGL_NO_SYNC_KHR) {
            // Insert a command into the GPU pipeline to wait
            //  until this fence is signaled (Non-blocking).
            // eglWaitSyncKHR does not block the CPU;
            //  it only inserts a dependency into the GPU command queue.
            eglWaitSyncKHR(dpy, sync, 0);

            // The sync object can be destroyed immediately;
            // the GPU internal reference is maintained.
            eglDestroySyncKHR(dpy, sync);
        }
        // [Important] Ownership of the fd is transferred to eglCreateSyncKHR.
        // While driver implementations may vary,
        //  the original fd is typically closed after creation.
        close(fence_fd);
    }

    // 2. Existing Rendering Logic (Zero-copy binding)
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_EXTERNAL_OES, camera_tex_ids[buffer_index]);

    glViewport(0, 0, w, h);
    glUseProgram(tex->program);

    // Set Vertex Attribute
    glEnableVertexAttribArray(0);
    glEnableVertexAttribArray(1);
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE,
      5 * sizeof(GLfloat), vVertices_vin_v_flip);
    glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE,
      5 * sizeof(GLfloat), &vVertices_vin_v_flip[3]);

    // 3. Execute Draw
    glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_SHORT, indices);

    // 4. (Optional) Create GPU Completion Fence (Out-fence)
    // If needed, an Out-fence can be created
    //   via eglCreateSyncKHR(..., EGL_SYNC_NATIVE_FENCE_ANDROID, NULL)
    // to notify the next stage (e.g., Display/DRM) that GPU rendering is complete.

    //if you done every opengl rendering, execute eglSwapBuffers()
    //    eglSwapBuffers(dpy, eglGetCurrentSurface(EGL_DRAW));
}
```

4 DRM direct method

If you do not need to do something on Camera Video, and just want to draw, we can directly send camera video frame to DRM plane.

DRM	Linux kernel subsystem for managing GPUs and display hardware. Consist of two main parts are KMS and GEM.
KMS	Configure display resolution and refreshment.
GEM	Manage GPU memory buffers. It allows the kernel to allocate, manage, and share memory buffers (GEM objects) between applications and the GPU. It is often used alongside DMA-BUF for secure, zero-copy buffer sharing. If you want scan-out without GPU, create DUMB buffer.

connector	Physical Port, for example HDMI and LVDS.
encoder	Translator between CRTC and Connectors.
crtc	The scan-out engine.
plane	A memory object. It feeds pixel data from a framebuffer to CRTC

DRM has 4 method to update output.

drmMode(Function)	Description
PageFlip	faster, but it is not supported in all DRM drivers.
SetCrtc	slower, but it is supported in all DRM drivers.
SetPlane	it is more flexible, but it is not supported in all DRM drivers.
Atomic commit	it is more flexible, but it is more complex to use. can be used if you need to do more complex operations, such as changing multiple properties at once, or if you need to do a non-blocking commit.

Assign Plane

Plane 0	As a primary Plan, assign to OpenGL (GPU)
Plane1	Assign for camera video output. If you need OSD, use alpha blend on OpenGL (GPU) and then adjust the zpos. If you need 4CH camera output, use extra Plane 2~4.

4.1 How to use UYVY pixel format.

When you setup FrameBuffer, use DRM_FORMAT_UYVY.

```
drmModeAddFB2(drm->fd, 1280, 720, DRM_FORMAT_UYVY,
              handles, pitches, offsets,
              &camera_fb_id, 0);
```

4.2 Atomic commit

Use drmModeAtomicAddProperty() and then execute drmModeAtomicCommit()

```
  drmModeAtomicReqPtr req = drmModeAtomicAlloc();
  drmModeAtomicAddProperty(req, plane0_id,..);
 ..
  ret = drmModeAtomicCommit(drm->fd, req, flags, &waiting_for_flip);
```

```
  drmModeAtomicFree(req);
```

4.3 Scale and display position

Scale down four 1280x720 channels to 960x540, and output them at 1920x1080.
To draw four 1280x720 video frame on 1920x1080 output panel.

- Plane 1: (0, 0, 960, 540)
- Plane 2: (960, 0, 960, 540)
- Plane 3: (0, 540, 960, 540)
- Plane 4: (960, 540, 960, 540)

```
// Example for Plane1~4
for (int i = 0; i < 4; i++) {
    uint32_t x = (i % 2) * 960;
    uint32_t y = (i / 2) * 540;
    uint32_plane_id = i+1;
    drmModeAtomicAddProperty(req, plane_id, props.fb_id, fbs[i]);
    drmModeAtomicAddProperty(req, plane_id, props.crtc_x, x);
    drmModeAtomicAddProperty(req, plane_id, props.crtc_y, y);
    drmModeAtomicAddProperty(req, plane_id, props.crtc_w, 960);
    drmModeAtomicAddProperty(req, plane_id, props.crtc_h, 540);
    //The Source Rect is fixed at the original 1280x720 resolution
    // (the hardware automatically performs the downscaling).
    drmModeAtomicAddProperty(req, plane_id, props.src_w, 1280 << 16);
    drmModeAtomicAddProperty(req, plane_id, props.src_h, 720 << 16);
}
```

4.4 How to support 30Fps camera and 60fps GPU output

Check if there are any frames generated by the camera in the VideoQueue, and then execute AddProperty().
If the output is only for the GUI, the resource previously allocated to Plane 1 is reused.

```
  drmModeAtomicReqPtr req = drmModeAtomicAlloc();
  drmModeAtomicAddProperty(req, plane0_id,..);
  ..
  if(camera_input) {
    drmModeAtomicAddProperty(req, plane1_id,..);
    ..
  }
  ret = drmModeAtomicCommit(drm->fd, req, flags, &waiting_for_flip);
  drmModeAtomicFree(req);
  ..
```

If you turn-off camera, assign 0 on fb_id and crtc_id.
For example

```
  drmModeAtomicAddProperty(req, plane_1_id, drm->props.fb_id_prop, 0);
  drmModeAtomicAddProperty(req, plane_1_id, drm->props.crtc_id_prop, 0);
  ..
  ret = drmModeAtomicCommit(drm_fd, req, flags, &waiting_for_flip);
```

4.5 wait vBlank

```
  uint32_t flags = DRM_MODE_ATOMIC_NONBLOCK | DRM_MODE_PAGE_FLIP_EVENT;
  ret = drmModeAtomicCommit(drm->fd, req, flags, &waiting_for_flip);
  drmModeAtomicFree(req);
  if(fence_fd)
    close(fence_fd);

  //wait event
  while (waiting_for_flip) {
    ret = select(drm->fd + 1, &drm->fds, NULL, NULL, NULL);
    if (ret <= 0) break;
    drmHandleEvent(drm->fd, &evctx);
  }
```

4.6 TEST_ONLY

To check the applied property, use TEST_ONLY flags to validate. If it is successes, execute drmModeAtomicCommit() without *DRM_MODE_ATOMIC_TEST_ONLY flag*.

```
int flags = DRM_MODE_ATOMIC_TEST_ONLY;
int ret = drmModeAtomicCommit(fd, atomic_req, flags, user_data);

if (ret == 0) {
    // setting is valid. To apply real properties,
    //  execute AtomicCommit() without DRM_MODE_ATOMIC_TEST_ONLY.
    flags = DRM_MODE_ATOMIC_NONBLOCK
    drmModeAtomicCommit(fd, atomic_req, flags, user_data);

} else {
    //failed,
}
```

4.7 with DMABUF

Change dmabuf_fd to DRM handle.

```
uint32_t handle;
// Change DMABUF_FD to DRM handle
drmPrimeFDToHandle(drm->fd,  dmabuf_fd,  &handle);
```

Use it when you add FrameBuffer.

```
uint32_t camera_fb_id;

uint32_t fb_id;
uint32_t handles[4] = {handle};
uint32_t pitches[4] = {1280 * 2}; // UYVY: width * 2
uint32_t offsets[4] = {0};
```

```
drmModeAddFB2(drm->fd, 1280, 720, DRM_FORMAT_UYVY,
              handles, pitches, offsets,
              &camera_fb_id, 0);
```

The following example is to generate FBs corresponding to the number of frames used in VIN.

```
#define NUM_OF_FRAMES 6

int create_framebuffer(int cam_idx)
{
  VideoQueue_t *vqueue;
  uint32_t handle;
  uint32_t camera_fb_id;
  uint32_t handles[4] = {0};
  uint32_t pitches[4] = {1280 * 2}; // 2 Bytes per Pixel on UYVY
  uint32_t offsets[4] = {0};


  for(int i=0; i < NUM_OF_FRAMES; i++) {

    vqueue = get_vin_vqueue(cam_idx, i); /* TODO */;
    drmPrimeFDToHandle(drm->fd,  vqueue->dmabuf_fd,  &handle);
    handles[0] = handle;
    drmModeAddFB2(drm->fd, 1280, 720, DRM_FORMAT_UYVY,
                handles, pitches, offsets, &camera_fb_id, 0);
    vqueue->drm_fb_id = camera_fb_id;
  }
}
```

4.8 Atomic Commit with fence

Below example merged the methods that we learned before.

```
int drm_update_atomic()
{
  drmEventContext evctx = {
    .version = DRM_EVENT_CONTEXT_VERSION,
    .page_flip_handler = page_flip_handler,
  };
  int fence_fd = 0;

  next_bo = gbm_surface_lock_front_buffer(drm->gbm_surf);
  fb = drm_fb_get_from_bo(drm, next_bo);

  drmModeAtomicReqPtr req = drmModeAtomicAlloc();

  drmModeAtomicAddProperty(req, plane0_id, props.fb_id_prop, fb->fb_id);
  drmModeAtomicAddProperty(req, plane0_id, props.crtc_id_prop, drm->crtc_id);

  drmModeAtomicAddProperty(req, plane0_id, props.crtc_x_prop, 0);
  drmModeAtomicAddProperty(req, plane0_id, props.crtc_y_prop, 0);
  drmModeAtomicAddProperty(req, plane0_id, props.crtc_w_prop, 1920);
  drmModeAtomicAddProperty(req, plane0_id, props.crtc_h_prop, 1080);
  drmModeAtomicAddProperty(req, plane0_id, props.src_w_prop, 1920 << 16);
```

```
  drmModeAtomicAddProperty(req, plane0_id, props.src_h_prop, 1080 << 16);

  if(vqueue_empty() != true) {
    vqueue = vqueue_pop();
    /* update camera video frame on plane1 */

    drmModeAtomicAddProperty(req, plane1_id, props.fb_id_prop, fb->fb_id);
    drmModeAtomicAddProperty(req, plane1_id, props.crtc_id_prop, drm->crtc_id);

    drmModeAtomicAddProperty(req, plane1_id, props.crtc_x_prop, 0);
    drmModeAtomicAddProperty(req, plane1_id, props.crtc_y_prop, 0);
    drmModeAtomicAddProperty(req, plane1_id, props.crtc_w_prop, 1280);
    drmModeAtomicAddProperty(req, plane1_id, props.crtc_h_prop, 720);
    drmModeAtomicAddProperty(req, plane1_id, props.src_w_prop, 1280 << 16);
    drmModeAtomicAddProperty(req, plane1_id, props.src_h_prop, 720 << 16);

    fence_fd = vqueue->fence_fd;
    if(fence_fd)
      drmModeAtomicAddProperty(req, plane_id1,
        props.in_fence_prop, (uint64_t)fence_fd);
  }

  uint32_t flags = DRM_MODE_ATOMIC_NONBLOCK | DRM_MODE_PAGE_FLIP_EVENT;
  ret = drmModeAtomicCommit(drm->fd, req, flags, &waiting_for_flip);
  drmModeAtomicFree(req);
  if(fence_fd)
    close(fence_fd);

  //wait event
  while (waiting_for_flip) {
    // reuse fds that was assigned on drm_init_select().
    ret = select(drm->fd + 1, &drm->fds, NULL, NULL, NULL);
    if (ret <= 0) break;
    drmHandleEvent(drm->fd, &evctx);
  }

  //replace buffer
  gbm_surface_release_buffer(drm->gbm_surf, drm->bo);
  drm->bo = next_bo;

  return 0;
}
```

4.9 COLOR_ENCODING and COLOR_RANGE

Step1. Find Property ID and Enum Value

First, you must find to COLOR_ENCODING and COLOR_RANGE on the Plane's properties to identify the IDs, as well as the integer values for your desired standards (e.g., BT.709, Full Range).

```
uint32_t encoding_prop_id = 0;
```

```
uint32_t range_prop_id = 0;
uint64_t bt709_val = 0;
uint64_t full_range_val = 0;

drmModeObjectProperties *props = drmModeObjectGetProperties(drm_fd, plane_id,
DRM_MODE_OBJECT_PLANE);
for (int i = 0; i < props->count_props; i++) {
    drmModePropertyPtr prop = drmModeGetProperty(drm_fd, props->props[i]);

    if (strcmp(prop->name, "COLOR_ENCODING") == 0) {
        encoding_prop_id = prop->prop_id;
        for (int j = 0; j < prop->count_enums; j++) {
            if (strcmp(prop->enums[j].name, "ITU-R BT.709 YCbCr") == 0)
                bt709_val = prop->enums[j].value;
        }
    } else if (strcmp(prop->name, "COLOR_RANGE") == 0) {
        range_prop_id = prop->prop_id;
        for (int j = 0; j < prop->count_enums; j++) {
            if (strcmp(prop->enums[j].name, "YCbCr full range") == 0)
                full_range_val = prop->enums[j].value;
        }
    }
    drmModeFreeProperty(prop);
}
```

Step 2 Add Atomic Request

```
drmModeAtomicReq *req = drmModeAtomicAlloc();

// Basic property setting (FB, CRTC, location, …)
drmModeAtomicAddProperty(req, plane_1_id, fb_id_prop, fb_id);
// ...

// Set color encoding (ex BT.709)
if (encoding_prop_id) {
    drmModeAtomicAddProperty(req, plane_1_id, encoding_prop_id, bt709_val);
}

// Set color range (ex Full Range)
if (range_prop_id) {
    drmModeAtomicAddProperty(req, plane_1_id, range_prop_id, full_range_val);
}

drmModeAtomicCommit(drm_fd, req, DRM_MODE_ATOMIC_NONBLOCK, NULL);
drmModeAtomicFree(req);
```

5. multi-thread assign

The “cam-thread” that I implemented, using 3 threads.

main()	parent
VIN thread	camera capture thread
DISPLAY thread	Display to Wayland or DRM. Implemented on OpenGL ES
IMR thread	For wide lens distortion correction. Located between VIN and DISPLAY thread.

As far as I knew, the thread context scheduling time was 25ms; therefore, I was curious as to how the three threads mentioned above were functioning correctly.
The reason is that the R-Car H3 features four Cortex-A57 cores, four Cortex-A53 cores, and two Cortex-R7 cores dedicated to real-time processing; since each thread operates on a separate CPU, they remain unaffected by context switching.
example to manually assign thread to CPU

```
#include <sched.h>

cpu_set_t cpuset;
CPU_ZERO(&cpuset);
CPU_SET(4, &cpuset); // Pin to a "big" core

pthread_t thread = pthread_self();
pthread_setaffinity_np(thread, sizeof(cpu_set_t), &cpuset);
```

The number 4~7 is a big core on R-CAR H3

```
#define _GNU_SOURCE
#include <sched.h>
#include <pthread.h>
#include <stdio.h>

void pin_to_big_cores()
{
    cpu_set_t cpuset;

    // 1. Start with a clean slate (All bits 0)
    CPU_ZERO(&cpuset);

    // 2. Add specific "Big" cores to the mask
    CPU_SET(4, &cpuset);
    CPU_SET(5, &cpuset);

    // 3. Apply the mask to the current thread
    pthread_t current_thread = pthread_self();
    int result = pthread_setaffinity_np(current_thread,
      sizeof(cpu_set_t), &cpuset);

    if (result != 0) {
        perror("pthread_setaffinity_np");
    } else {
        printf("Thread pinned to CPUs 4 and 5.\n");
    }
}
```

6. Static Video Queue

To manage the “video queue,” the existing “cam-thread” utilized `malloc()` and `free()`. These functions, by continuously allocate and free heap memory, give rise to the problem of memory fragmentation. In systems that require stability and long-term operation, the use of `malloc()` and `free()` is generally prohibited; furthermore, MISRA C—the standard adopted by automotive companies—explicitly prohibits their use.

6.1 current code

This code was using a modified version of GMiniObject, which was originally used in GStreamer.

```
typedef struct meta_s {
    int cam_idx;    //OLD camera_id
    int buf_idx;    //OLD:index
    int width, height;
    u32 pixelformat;

    void *plane[3];
    int dmafd[3];
    u32 offset[3];
    u32 stride[3];
    int image_size;

    void *texture;  //OLD priv. for Wayland/Weston interface.
    int flags;      //GST_META_FLAG_POOLED
    void *vsp_mem;  //OLD:priv for vsp_mem_t. sv->meta_plane[][], sv->alpha_plane[][]
                    //car use priv to read car image.

    void *pool;     /* to container. ex:imr_buffer_t */
} meta_t;
```

```
typedef struct GBuffer_s GBuffer;
struct GBuffer_s {
    /* replace GMiniObject mini_object */
    void *owner; /* OLD:pool. dispose function owner. ex:(struct vin_data *) */
    int (*dispose)(void *arg); /* for VIN qbuf */
    int refcount; /* for g_buffer_ref() and gst_buffer_unref() */

    void *destructor;           //TDB
    void (*destroy)(void *arg); //TDB

    /* replace VMETA and META */
    meta_t meta;

    /* queue (or list) */
    GBuffer *prev;
    GBuffer *next;
    char name[32];  /* for debug */
    int free;       /* if 1, and dispose() return true, free this buffer. */

    /* timestamp */
    struct timeval pts;
    struct timeval dts;
    struct timeval duration;
```

```
};
```

Below is an example that we needed so far.

```
typedef struct vqueue_s {
    uint32_t cam_idx; /* 0~3 */
    uint32_t buf_idx; /* 0~5 */
    void *plane[2]; /* for mmap */
    uint32_t dbuf_fd[2]; /* for expbuf with DMABUF */
    uint32_t fence_fd[2]; /* for sync fence */
    uint32_t offset[2];
    uint32_t stride[2];
    uint32_t image_size;

    void *dispose_owner; /* for VIN qbuf */
    int (*dispose)(void *arg); /* for VIN qbuf */
    struct vqueue_s *next;
} vqueue_t;

typedef struct vqueue_header_s {
    pthread_mutex_t lock;
    pthread_cond_t wait;
    pthread_cond_t cond_not_empty;
    pthread_cond_t cond_not_full;
    vqueue_t *front;
    vqueue_t *rear;
    int count; /* content counter */
} vqueue_header_t;
```

And below is new structure that is considered aligment.

```
typedef struct vqueue_s {
    /* 8-byte members: move-up and remove padding */
    void *plane[2];             /* 16 bytes */
    void *dispose_owner;        /* 8 bytes (for VIN qbuf) */
    int (*dispose)(void *arg);  /* 8 bytes (for VIN qbuf) */
    struct vqueue_s *next;      /* 8 bytes */

    /* 4-byte members: located with 8bytes alignment */
    uint32_t cam_idx;           /* 4 bytes (0~3) */
    uint32_t buf_idx;           /* 4 bytes (0~5) */

    uint32_t dbuf_fd[2];        /* 8 bytes (for expbuf with DMABUF) */
    uint32_t fence_fd[2];       /* 8 bytes (for sync fence) */

    uint32_t offset[2];         /* 8 bytes */
    uint32_t stride[2];         /* 8 bytes */

    uint32_t image_size;        /* 4 bytes */
    uint32_t reserved;          /* 4 bytes: add padd for multiple 8 alignmant */
} vqueue_t;
```

```
typedef struct vqueue_header_s {
    pthread_mutex_t lock;       /* 40 bytes (on 64-bit Linux) */
    pthread_cond_t wait;        /* 48 bytes */
    pthread_cond_t cond_not_empty;
    pthread_cond_t cond_not_full;

    /* 8-byte pointers */
```

```
    vqueue_t *front;            /* 8 bytes */
    vqueue_t *rear;             /* 8 bytes */

    /* 4-byte member */
    int count;                  /* 4 bytes */
    uint32_t reserved;          /* 4 bytes: Alignment padding */
} vqueue_header_t;
```

It uses VQUEUE_POOL_DYNAMIC to demote the dynamic allocation.
We will use a static allocation without VQUEUE_POOL_DYNAMIC.
And, add all available items on vqueue_pool_header at the beginning.

```
#define VQUEUE_POOL_SIZE 128

#ifdef VQUEUE_POOL_DYNAMIC
    /* use dynamic allocation (malloc and free)*/
#else
    /* use static allocation */
  vqueue_t vqueue_pool[VQUEUE_POOL_SIZE]; /* global pool */
#endif

vqueue_header_t vqueue_pool_header; /* global pool header */
```

```
int vqueue_push_rear(vqueue_header_t *header, vqueue_t *item);
int vqueue_pop_front(vqueue_header_t *header, vqueue_t *item);
bool vqueue_is_empty(vqueue_header_t *header);
int vqueue_pop_all(vqueue_header_t *header);
int vqueue_header_init(vqueue_header_t *header);
int vqueue_header_destroy(vqueue_header_t *header);

vqueue_t *vqueue_pool_get(void);
int vqueue_pool_put(vqueue_t *item);
int vqueue_pool_create(void);
int vqueue_pool_destroy(void);
```

So, we need a vqueue_pool_create() before we use it, and vqueue_pool_destroy() at the end of main().

```
main()
{
 vqueue_pool_create();
 ..
 vqueue_pool_destroy();
}
```

In VIN that has 4 output queue, we alos need a `vqueue_header_init()` and `vqueue_header_destroy()`.

```
vqueue_header_t vin_output_vqueue[4];

for(int i=0; i < 4; i++)
  vqueue_header_init(&vin_output_vqueue[i]);
…
for(int i=0; i < 4; i++)
  vqueue_header_destroy(&vin_output_vqueue[i]);
```

And put video frame on vin_output_vqueue when we executes DQUEUE, and send it to DISPLAY when we have all video channel inputs.

```
vin_process_buffer(struct vin_data *vin, int cam_idx)
{
  ret = vin_qdbuf(vin, cam_idx, &buf_idx, NULL, NULL);
  buffer = &vdev->pool[buf_idx];

  vqueue_t *item = vqueue_pool_get(); /* get new item */
  item->cam_idx = cam_idx;
  item->buf_idx = buf_idx;
  item->plane[0] = buffer->plane[0];
  item->dbuf_fd[0] = buffer->dmafd[0]
  item->fence_fd[0] = buffer->fence_fd[0];
  item->offset[0] = 0;
  item->stride[0] = 0;
  item->image_size = buffer->image_size;
  item->dispose_owner = (void *)vin;
  item->dispose = vin_dispose_buffer;
  vqueue_push_rear((&vin_output_vqueue[cam_idx], item);

  ..
  if(ready_all_channels) {
    for(int i=0; i < 4; i++)
      item = vqueue_pop_front(((&vin_output_vqueue[i]);
      vqueue_push_rear(&display_vin_vqueue);
      // if vin_output_vqueue[i] is not empty,
      //    dispose it.
      //     if it had fence_fd, close it.
  }
}
```

When you used item, and no more needed, use `vqueue_pool_put() to move back to vqueue_pool.`

Q&A

Q: Why it using 6 frame buffers for VIN?
R-CAR uses 3 Hardware slots. The minimum requirement is 4. OpenGL or DRM uses 2 frames, one for update and the other for scan-out. The extra one is used for VideoQueue processing, and mandatory needed to support 4CH video sync.

Q: Why it using 6 frames on DMABUF?
This is because **pipeline stability (jitter-free)** is far more important than saving a small amount of memory.

Q: If use two threads for multi-panels, what is needed?
We are using one GPU. use makeCurrent().